

MoleNet v7.1 LoRaWAN Setup Guide

Part A - Connect MoleNet Board to LoRaWAN

Step 1. Confirm LoRa Hardware

Check the MoleNet v7.1 schematic and confirm the LoRa module.

- Module: RA-01SH
- Chip: SX1262
- Driver file: SX1262.py

Use SX1262.py as the driver for the LoRa module.

Step 2. Install Thonny

Download and install Thonny on your laptop from:

```
thonny.org
```

Thonny is used to flash MicroPython, upload files to the MoleNet board, and run the program.

Step 3. Flash MicroPython

Flash MicroPython for ESP32-S3 to the MoleNet board.

In Thonny:

- Go to Tools -> Options -> Interpreter
- Choose MicroPython (ESP32)
- Select the correct COM port
- Click Install or update MicroPython if needed
- Select the ESP32_GENERIC_S3 firmware

After flashing, the Thonny shell should show:

```
>>>
```

This means the board is ready to run MicroPython code.

Step 4. Prepare Files on Laptop

Keep these files together in one folder on your laptop:

- SX1262.py
- LoRaWAN.py
- EU868.py
- config_ABP.py
- cmac.py

Later, main.py will also be added to the same folder.

Step 5. Create TTN Account

Create or log in to an account at The Things Network using the TTN Console.

Use:

```
thethingsnetwork.org
```

Step 6. Create TTN Application

Open The Things Network Console.

Go to:

```
Applications -> Add application
```

Create a new application for the MoleNet device.

Use a clear application name, for example:

```
molenet-test
```

Step 7. Register End Device

Register the device manually with the following settings:

- Frequency plan: Europe 863-870 MHz
- LoRaWAN version: 1.0.4
- Regional Parameters version: RP002 1.0.4
- Activation mode: ABP

Use ABP for this simple LoRaWAN connection test.

Step 8. Generate TTN Keys

Generate and copy the following values from TTN:

- DevEUI
- DevAddr
- AppSKey
- NwkSKey

These values are needed in config_ABP.py.

Step 9. Add TTN Keys to config_ABP.py

Create or open the file:

```
config_ABP.py
```

Paste the TTN values into config_ABP.py. A simple format is:

```
import ubinascii

DevEUI = ubinascii.unhexlify("PASTE_DEVEUI_HERE")
DevAddr = ubinascii.unhexlify("PASTE_DEVADDR_HERE")
AppSKey = ubinascii.unhexlify("PASTE_APPSKEY_HERE")
NwkSKey = ubinascii.unhexlify("PASTE_NWKSKEY_HERE")
```

Example for DevAddr:

```
DevAddr = ubinascii.unhexlify("260B304C")
```

Using `ubinascii.unhexlify()` is easier than manually writing byte arrays such as:

```
DevAddr = [0x26, 0x0B, 0x30, 0x4C]
```

Make sure there are no spaces inside the key strings.

Step 10. Write main.py

Create a file named `main.py` and paste the following code:

```
from machine import SoftSPI, Pin
from SX1262 import Transceiver
from LoRaWAN import LoRaWAN
from config_ABP import NwkSKey, AppSKey, DevAddr, DevEUI
import EU868
import utime

utime.sleep_ms(200)

spi = SoftSPI(
    baudrate=100000,
    sck=Pin(14),
    mosi=Pin(47),
    miso=Pin(21)
)

cs = Pin(48, Pin.OUT, value=1)
rst = Pin(15, Pin.OUT, value=1)
busy = Pin(39, Pin.IN)
dio1 = Pin(46, Pin.IN)

sx = Transceiver(spi, cs, rst, busy, dio1)
lw = LoRaWAN(sx, EU868.FREQS)

lw.setup_abp(NwkSKey, AppSKey, DevAddr, DevEUI)

print("LoRaWAN ready")

while True:
    try:
        lw.send("hello", power=17, sf=7)
        print("sent")
    except Exception as e:
        print("send error:", e)

    utime.sleep(60)
```

This code sends the message "hello" every 60 seconds.

Step 11. SX1262 Pin Mapping

The working SX1262 pin mapping for MoleNet v7.1 is:

- SCK = GPIO14
- MOSI = GPIO47
- MISO = GPIO21
- CS = GPIO48
- RST = GPIO15
- BUSY = GPIO39
- DIO1 = GPIO46

Use these pins in the LoRaWAN code.

Part B - Upload Files via Thonny

Step 12. Open File Pane

In Thonny, click:

```
View -> Files
```

The file window shows two sides:

- Left side: laptop file system
- Right side: MicroPython device

Step 13. Open Your Laptop Folder

In the left pane, navigate to the folder containing:

- SX1262.py
- LoRaWAN.py
- EU868.py
- config_ABP.py
- cmac.py
- main.py

Step 14. Upload Files to the Board

For each file, right-click it in the left pane and select:

```
Upload to /
```

Upload all six files:

- SX1262.py
- LoRaWAN.py
- EU868.py
- config_ABP.py
- cmac.py
- main.py

Step 15. Verify Uploaded Files

In the right panel under the MicroPython device, confirm that all six files are present.

Step 16. Run main.py

In Thonny:

- Double-click main.py on the board side
- Click the green Run button

Step 17. Confirm in Thonny Shell

The Thonny shell should output:

```
LoRaWAN ready  
sent
```

If this appears, the program is running and the board is trying to send LoRaWAN packets.

Step 18. Confirm in TTN

In The Things Network Console, go to:

```
Applications -> your application -> your device -> Live data
```

Confirm that TTN shows a new uplink message.

For the "hello" test, the payload will be:

```
68 65 6C 6C 6F
```

This decodes to:

```
hello
```

At this point, the basic LoRaWAN connection is complete.

For a simple test, checking the raw payload is enough. For real sensor data, create a payload formatter in TTN. A payload formatter converts the raw payload into readable fields such as temperature, humidity, pressure, and soil moisture.

Part C - Send TTN Data to Google Sheets

Step 19. Create Google Sheet

Create a new Google Sheet.

Rename the bottom sheet tab to:

```
data
```

In row 1, add the following column headers:

- time
- device_id
- temp_c
- humidity_pct
- raw_json

Step 20. Open Apps Script

From the Google Sheet, click:

Extensions -> Apps Script

This opens the Google Apps Script editor.

Step 21. Paste Apps Script Code

Delete the default code in Apps Script and paste the following code:

```
function doGet() {
  return ContentService
    .createTextOutput("Webhook is alive")
    .setMimeType(ContentService.MimeType.TEXT);
}

function doPost(e) {
  try {
    const SPREADSHEET_ID = "YOUR_SPREADSHEET_ID";
    const SHEET_NAME = "data";

    const ss = SpreadsheetApp.openById(SPREADSHEET_ID);
    const sheet = ss.getSheetByName(SHEET_NAME);

    if (!sheet) {
      throw new Error('Sheet "data" not found');
    }

    const body = JSON.parse(e.postData.contents);

    const deviceId =
      body.end_device_ids?.device_id ||
      "";

    const time =
      body.uplink_message?.received_at ||
      body.received_at ||
      new Date().toISOString();

    const decoded =
      body.uplink_message?.decoded_payload ||
      {};

    const temp =
      decoded.temp_c ??
      decoded.dht_temp_c ??
      "";

    const hum =
      decoded.humidity_pct ??
      decoded.dht_humidity_pct ??
      "";

    sheet.appendRow([
      time,
      deviceId,
      temp,
      hum,
      JSON.stringify(body)
    ]);
  }
}
```

```

    });

    return ContentService
      .createTextOutput(JSON.stringify({ status: "ok" }))
      .setMimeType(ContentService.MimeType.JSON);

  } catch (err) {
    return ContentService
      .createTextOutput(JSON.stringify({
        status: "error",
        message: String(err)
      }))
      .setMimeType(ContentService.MimeType.JSON);
  }
}

```

This script saves the received time, device ID, decoded temperature, decoded humidity, and the full raw TTN JSON message. The raw JSON is important because the exact field names depend on the TTN payload formatter.

Step 22. Replace Spreadsheet ID

In the Apps Script code, replace:

```
YOUR_SPREADSHEET_ID
```

with the actual Spreadsheet ID from the Google Sheet URL.

Example Google Sheet URL:

```
https://docs.google.com/spreadsheets/d/1abcDEFghiJKLmnop123456789/edit
```

The Spreadsheet ID is the part between /d/ and /edit.

```
1abcDEFghiJKLmnop123456789
```

So this line:

```
const SPREADSHEET_ID = "YOUR_SPREADSHEET_ID";
```

should become something like:

```
const SPREADSHEET_ID = "1abcDEFghiJKLmnop123456789";
```

Step 23. Save Apps Script

Save the project using Ctrl + S or click the save icon.

Step 24. Deploy as Web App

In Apps Script:

- Click Deploy -> New deployment
- Choose Web app
- Set Execute as = Me
- Set Who has access = Anyone
- Click Deploy

- Authorize the project if prompted

After deployment, Google gives a Web App URL.

Important: If the Apps Script code is edited later, save the code and deploy the Web App again. Saving alone is not enough. TTN will continue using the old deployed version unless the script is deployed again.

Step 25. Copy Web App URL

Copy the Web App URL. It should end with:

```
/exec
```

Test the URL in a browser. It should display:

```
Webhook is alive
```

If this message appears, the Web App is active.

Step 26. Add TTN Webhook

In The Things Network Console, go to:

```
Applications -> your application -> Integrations -> Webhooks
```

Then select:

```
Add webhook -> Custom webhook
```

Enter the following settings:

- Webhook ID: google-sheets
- Base URL: paste the Apps Script /exec URL
- Enable/select: Uplink message

Do not enter a custom uplink path unless the TTN form specifically asks for one. The important point is that the webhook must be enabled for uplink messages.

Save the webhook.

Step 27. Reactivate if Needed

If TTN shows the webhook as deactivated, click Reactivate. Then send another uplink message from the board.

Step 28. Send a New Packet

Let the board send another packet. For the test code, wait for the next 60 second cycle. You can also press reset on the board to restart the program and send again.

Step 29. Verify in Google Sheet

A new row should appear in the data sheet.

The row should contain:

- time
- device_id
- temp_c
- humidity_pct
- raw_json

If raw_json is filled but temp_c and humidity_pct are empty, the webhook is working. In that case, the issue is that the TTN payload formatter or the field names in Apps Script do not match the actual decoded payload. Check the raw_json column and adjust the Apps Script field names if needed.

Part D - Payload Formatter Note

For the first LoRaWAN test, the board sends only:

```
hello
```

This confirms that the LoRaWAN connection works.

For real sensor data, use a TTN payload formatter. The payload formatter should decode the raw payload into readable fields.

Example decoded fields can be:

- temp_c
- humidity_pct
- pressure_hpa
- soil1_moisture
- soil1_temp
- soil2_moisture
- soil2_temp

After creating or editing a payload formatter in TTN, send a new uplink message and check the TTN Live Data page. If TTN shows decoded_payload, then Google Sheets can read those decoded values.

Part E - Debugging Google Sheets Connection

If Google Sheets does not update, check these steps.

1. Test the Web App URL

Open the Apps Script Web App URL in a browser. It should show:

```
Webhook is alive
```

If it does not show this, the Apps Script deployment is not correct.

2. Check TTN Live Data

Go to TTN Live Data and confirm that uplink messages are arriving. If TTN does not receive uplinks, the problem is in the LoRaWAN connection, not in Google Sheets.

3. Check Apps Script Executions

In Apps Script, open Executions. Check whether the script receives requests. If there is an error, open the failed execution and read the error message.

4. Redeploy After Editing

If the Apps Script code was changed, deploy the Web App again.

Use:

```
Deploy -> Manage deployments -> Edit -> New version -> Deploy
```

Saving the code is not enough.

5. Check the raw_json Column

If a row appears in Google Sheets but some decoded fields are empty, check the raw_json column. The full TTN message is stored there. Use it to find the correct field names, then update the Apps Script code.

6. Optional: Test with Postman

If needed, test the Apps Script Web App URL using Postman. Send a POST request to the Web App /exec URL with a sample TTN JSON body. This helps check whether the Apps Script works without waiting for a real LoRaWAN packet.

Setup Complete

The setup is complete when all of the following are true:

- The board sends packets from Thonny.
- TTN Live Data shows uplink messages.
- Google Sheets receives new rows.
- The raw_json column is filled.
- Decoded fields appear if a TTN payload formatter is used.

After this, the MoleNet v7.1 LoRaWAN setup is ready for sensor-data testing.